DAY 30

Databricks Interview Rapid-Fire

10 questions every Databricks candidate must answer without hesitation

This is the closing document of the 30-day series. The reel gave you ten crisp one-line answers. This document gives you the depth behind each one: the mechanism, the follow-up questions an interviewer will ask, and the trap that separates a memorized answer from an understood one.

01 Why Rapid-Fire Answers Win Interviews

Interviewers rarely want a lecture. They want to know in thirty seconds whether you understand the idea or have only read the vocabulary. A strong answer has three layers, and the rest of this document is written in that shape.

- **Layer 1, the definition.** One sentence that states what the thing is and what problem it solves.
- **Layer 2, the mechanism.** Two or three sentences on how it works internally.
- **Layer 3, the catch.** The limitation, trade-off or common mistake. This is what signals real experience.

INTERVIEW TIP

Always finish with the catch. Anyone can say VACUUM deletes old files. Only someone who has used it knows that it also limits how far back time travel can go.

The ten questions at a glance

#	Topic	One-line answer
1	Control vs Compute plane	Control plane runs in Databricks' account; compute plane runs in your cloud account and processes your data.
2	Delta Lake	Parquet files plus a transaction log, giving ACID, time travel and schema enforcement.
3	Unity Catalog	Central governance: access control, lineage, audit, with catalog.schema.table naming.
4	Medallion	Bronze raw, Silver cleaned, Gold business-ready.
5	OPTIMIZE vs VACUUM	OPTIMIZE compacts small files; VACUUM removes unreferenced old files.
6	Liquid Clustering	Flexible replacement for partitioning and Z-order; keys can be changed later.
7	Auto Loader	Incremental, scalable ingestion of new files from cloud storage.
8	Streaming exactly-once	Checkpoints plus replayable sources and idempotent sinks.
9	Declarative Pipelines	You declare the datasets; the platform handles dependencies and quality expectations.
10	Photon	Vectorized C++ engine that speeds up SQL and DataFrame workloads.

02 Platform and Governance (Q1, Q3, Q4)

Q1. Control plane vs Compute plane

The control plane is the backend managed by Databricks. It hosts the web application, notebooks, job scheduler, cluster manager and workspace configuration. The compute plane is where your data is actually processed. In the classic model, clusters run as virtual machines inside your own cloud account, and they read and write data in your own cloud storage. With serverless, the compute runs in a Databricks-managed account instead.

THE CATCH

The control plane stores metadata, notebook code and configuration. Your table data stays in your cloud storage. This is a very common follow-up question about security and data residency.

Q3. Unity Catalog

Unity Catalog is the unified governance layer for data and AI assets across workspaces. It gives you one place to define permissions, track lineage and capture audit logs. Objects are addressed with a three-level namespace: catalog, schema, and table (or view, volume, function, model).

```
SELECT * FROM sales_prod.orders.daily_summary;
GRANT SELECT ON TABLE sales_prod.orders.daily_summary
  TO `analysts`;
```

- **Access control.** Permissions are defined once at the account level and apply in every attached workspace.
- **Lineage and audit.** Column-level lineage is captured automatically, and audit logs record who accessed what.

Q4. Medallion architecture

Layer	What it holds	Typical operations
Bronze	Raw data exactly as ingested, with metadata such as source file and load time	Append-only ingestion, no business logic
Silver	Cleaned, deduplicated, validated and conformed data	Type casting, joins, deduplication, quality checks
Gold	Aggregated, business-ready tables for reporting and ML	Aggregations, dimensional models, feature tables

THE CATCH

Medallion is a design pattern, not a Databricks product. The interview trap is saying Bronze must always be a separate physical copy. What matters is the progressive improvement of quality, and that you can always reprocess from Bronze.

03 Delta Lake and Table Maintenance (Q2, Q5, Q6)

Q2. What is Delta Lake?

Delta Lake is an open table format. Data is stored as Parquet files, and a transaction log (the _delta_log folder) records every change as an ordered series of JSON commit files, with periodic checkpoint files. Because readers and writers consult the log rather than listing files, you get ACID transactions, snapshot isolation, time travel and schema enforcement on top of cheap object storage.

- **Concurrency.** Writers use optimistic concurrency control. If two writers conflict, one fails and retries.
- **Time travel.** Query an older version with VERSION AS OF or TIMESTAMP AS OF.
- **Schema enforcement.** A write whose schema does not match is rejected, unless schema evolution is explicitly enabled.

Q5. OPTIMIZE vs VACUUM

	OPTIMIZE	VACUUM
Purpose	Compacts many small files into fewer larger files	Deletes data files no longer referenced by the log and older than the retention period
Effect on speed	Faster reads	None directly; reduces storage cost
Effect on history	History preserved; old files are marked removed, not deleted	Removes the ability to time travel to versions that need the deleted files
Default retention	Not applicable	7 days

```
OPTIMIZE sales_prod.orders.daily_summary;

VACUUM sales_prod.orders.daily_summary RETAIN 168 HOURS;
```

THE CATCH

OPTIMIZE and VACUUM are different jobs. OPTIMIZE alone does not free storage, and VACUUM alone does not fix small files. Also, setting retention below the default risks breaking long-running readers and time travel.

Q6. Liquid Clustering

Liquid Clustering organizes data by the columns you choose so queries can skip files that cannot match a filter. It replaces both Hive-style partitioning and ZORDER for Delta tables. You declare the keys with CLUSTER BY, and you can change them later with ALTER TABLE without rewriting the entire table up front.

```
CREATE TABLE sales_prod.orders.events (
  event_id BIGINT, customer_id BIGINT, event_date DATE
) CLUSTER BY (customer_id, event_date);

-- Change keys later as query patterns evolve
ALTER TABLE sales_prod.orders.events CLUSTER BY (event_date);

OPTIMIZE sales_prod.orders.events;
```

THE CATCH

A table uses either Liquid Clustering or partitioning with Z-order, not both together. Data is clustered incrementally when OPTIMIZE runs, so keep running it (or use automatic clustering where available).

04 Ingestion and Streaming (Q7, Q8)

Q7. Auto Loader

Auto Loader incrementally and efficiently processes new files as they arrive in cloud storage. It is used through the cloudFiles source in Structured Streaming. It remembers which files it has already processed, so you never reprocess old data, and it can infer and evolve the schema.

```
df = (spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", "/Volumes/main/raw/_schemas/orders")
  .load("/Volumes/main/raw/orders/"))
```

Mode	How new files are found	Best for
Directory listing	Lists the input directory	Simple setups, low to moderate file volumes
File notification	Uses cloud event notifications and a queue	Very large directories and high file volumes

Q8. How does streaming achieve exactly-once?

Three pieces work together. First, the source must be replayable, so the engine can re-read data from a known offset. Second, the checkpoint records progress (offsets) and state, so a restarted query resumes exactly where it stopped. Third, the sink must be idempotent, so a replayed batch does not create duplicates. Delta Lake tables satisfy the sink requirement.

- **Default or processingTime trigger.** Runs continuously and processes data as it arrives.
- **AvailableNow trigger.** Processes everything available, in one or more batches, then stops. Ideal for scheduled incremental loads and it replaces the deprecated Trigger.Once.

THE CATCH

Never delete or share a checkpoint location casually. Each streaming query needs its own checkpoint, and losing it means the query loses its progress.

05 Transformation and Performance (Q9, Q10)

Q9. Lakeflow Spark Declarative Pipelines (formerly Delta Live Tables)

In a declarative pipeline you define what each dataset should contain, using SQL or Python, and the platform works out the dependency graph, ordering, retries, and incremental processing. Datasets are typically streaming tables (for incremental ingestion) and materialized views (for transformations and aggregations).

```
CREATE OR REFRESH STREAMING TABLE orders_silver (
  CONSTRAINT valid_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS SELECT * FROM STREAM(orders_bronze);
```

Expectation action	Behavior
Default (no ON VIOLATION)	Keeps the invalid rows and records the violation in metrics
ON VIOLATION DROP ROW	Drops invalid rows and records the count
ON VIOLATION FAIL UPDATE	Stops the update when any row violates the rule

NAMING NOTE

The product was originally called Delta Live Tables (DLT), then became Lakeflow Declarative Pipelines, and the documentation now uses Lakeflow Spark Declarative Pipelines. Interviewers may use any of these names, so recognize all of them.

Q10. Photon

Photon is a vectorized query engine written in C++. It processes data in columnar batches and makes better use of modern CPUs. It accelerates SQL and DataFrame operations such as scans, joins, aggregations and writes, and it requires no code changes. Spark's Catalyst optimizer still plans the query. Photon changes how supported operators are executed.

THE CATCH

Photon does not support every operation. When a part of the plan is unsupported, that part runs on the standard Spark engine, so the query still succeeds. Photon is also billed at a higher DBU rate on classic compute, so the speed-up should justify the cost.

How the pieces fit together

```
Source files --> Auto Loader --> Bronze --> Silver --> Gold --> SQL / ML
                                 \__ Declarative Pipelines __/
        Unity Catalog governs everything  |  Photon speeds it up
```

06 Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability: guarantees that make transactions reliable
Checkpoint	Stored progress and state of a streaming query, used for recovery
Expectation	A declared data quality rule inside a declarative pipeline
Idempotent sink	A destination where replaying the same write does not create duplicates
Lineage	The recorded path of data from source to every downstream object
Liquid Clustering	Adaptive data layout driven by CLUSTER BY keys
Materialized view	A dataset whose results are precomputed and refreshed by the platform
Medallion	Bronze, Silver, Gold layering pattern for progressive data quality
Photon	Databricks' native vectorized execution engine
Transaction log	The _delta_log record of every commit to a Delta table

07 Interview Questions and Answers

Q1. A Delta table query is slow because of thousands of tiny files. What do you do?

Run OPTIMIZE to compact files. For a long-term fix, find out why the files are small: frequent small appends or over-partitioning are the usual causes. Consider Liquid Clustering and automatic file size management instead of fine-grained partitions.

Q2. Why did time travel fail on a version from three weeks ago?

VACUUM probably removed the data files for that version, since the default retention is seven days. The log may still list the version, but the files are gone. If long history is a business need, increase delta.deletedFileRetentionDuration and log retention accordingly.

Q3. When would you choose file notification mode in Auto Loader?

When the input directory has a very large number of files or high arrival rates, because repeated directory listing becomes slow and expensive. File notification uses cloud events and a queue, at the cost of extra cloud setup and permissions.

Q4. How do you guarantee no duplicates if a streaming job crashes and restarts?

The checkpoint lets the query resume from the last committed offsets. Because the source is replayable and Delta commits are atomic and idempotent, a replayed batch does not double-write. For merge-style logic, use MERGE or foreachBatch with a deterministic key.

Q5. How is Unity Catalog different from the legacy Hive metastore?

Unity Catalog is account-level and spans workspaces, adds a third namespace level (catalog), and brings fine-grained access control, automatic lineage and audit. The legacy Hive metastore is workspace-scoped with much weaker governance.

Q6. Your pipeline must reject bad records but never stop. Which expectation action?

Use ON VIOLATION DROP ROW for the rule and monitor the dropped-row metrics. To keep bad records for review instead of losing them, route them into a quarantine table using the inverse condition. Use FAIL UPDATE only when bad data must never pass.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword DATABRICKS on any reel to get the next link in your DMs.

Piyush Varma | The Data Engineer Lab